Great — this is the **right instinct**. Below is your **same code**, but now each function has a **detailed, concept-level comment** that explains:

- **What problem the function solves**
- **Which pointers change**
- **What invariants must stay true (`head`, `tail`, `size`)**
- **What mistake to avoid**

No logic is changed. Only **thinking is added**.

```java
package Linkedlist.Revision;

public class PracticeLinkedList {
    Node head = null;
    Node tail = null;
    int size = 0;

    /*
     * Node represents a single element of the linked list.
     * Each node stores:
     * 1. data  → actual value
     * 2. next  → reference to the next node in the list
     *
     * The list is formed by chaining nodes using the `next` reference.
     */
    public static class Node {
        int data;
        Node next;

        public Node(int data) {
            this.data = data;
            next = null;
        }
    }

    /*
     * AddF (Add First):
     * Inserts a new node at the beginning of the list.
     *
     * Core idea:
     * - The new node must point to the current head.
     * - Only AFTER linking do we move head to the new node.
     *
     * Why order matters:
     * - Moving head first would lose access to the existing list.
     *
     * Edge case:
     * - If list is empty, both head and tail must point to the new node.
     */
    public void AddF(int data) {
```

```java
        Node newNode = new Node(data);

        if (head == null) {
            head = tail = newNode;
            size++;
            return;
        }

        newNode.next = head; // connect new node to existing list
        head = newNode;      // move head
        size++;
    }

    /*
     * RemoveF (Remove First):
     * Removes the node at the beginning of the list.
     *
     * Core idea:
     * - Removing the first node means moving head to head.next.
     *
     * Edge cases:
     * - Empty list → nothing to remove.
     * - Single node → both head and tail must become null.
     *
     * Invariant:
     * - After removal, head must always point to the first valid node.
     */
    public void RemoveF() {
        if (size == 0) {
            System.out.println("List is Empty");
            return;
        }

        if (size == 1) {
            head = tail = null;
        } else {
            head = head.next;
        }
        size--;
    }

    /*
     * AddAt (Add at Index):
     * Inserts a new node at a specific index.
     *
     * Core idea:
     * - To insert at index i, stop at node (i - 1).
     * - Rewire links so the new node fits between two nodes.
     *
     * Steps:
     * 1. Validate index.
     * 2. Traverse to node just BEFORE insertion index.
     * 3. First connect newNode.next to the remaining list.
     * 4. Then connect previous node to newNode.
```

```java
     *
     * Common mistake:
     * - Changing links in the wrong order breaks the chain.
     */
    public void AddAt(int index, int data) {

        if (index < 0 && index > size) {
            System.out.println("Invalid index");
            return;
        }

        if (index == 0) {
            AddL(data);
            return;
        }

        if (index == size) {
            AddL(data);
            return;
        }

        Node newNode = new Node(data);
        Node temp = head;

        // Traverse to node just before target index
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }

        // Preserve remaining list before insertion
        newNode.next = temp.next;
        temp.next = newNode;
        size++;
    }

    /*
     * RemoveAt (Remove at Index):
     * Deletes the node present at a specific index.
     *
     * Core idea:
     * - You NEVER directly delete a node.
     * - You remove it by changing the previous node’s `next` reference.
     *
     * Steps:
     * 1. Validate index.
     * 2. Handle edge cases (first and last node).
     * 3. Traverse to node just BEFORE the target.
     * 4. Skip the target node by reconnecting links.
     *
     * Invariant:
     * - After removal, list must remain fully connected.
     */
    public void RemoveAt(int index) {
        if (index < 0 || index >= size) {
```

```java
            System.out.println("Invalid Index");
            return;
        }

        if (index == 0) {
            RemoveF();
            return;
        }

        if (index == size - 1) {
            RemoveL();
            return;
        }

        Node temp = head;

        // Move to node just before the one to be removed
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }

        // Bypass the node at index
        temp.next = temp.next.next;
        size--;
    }

    /*
     * AddL (Add Last):
     * Appends a new node at the end of the list.
     *
     * Core idea:
     * - Use tail to achieve O(1) insertion at the end.
     *
     * Steps:
     * 1. Link current tail to new node.
     * 2. Move tail reference to the new node.
     *
     * Edge case:
     * - Empty list → head and tail must both point to new node.
     */
    public void AddL(int data) {
        Node newNode = new Node(data);

        if (head == null) {
            head = tail = newNode;
            size++;
            return;
        }

        tail.next = newNode;
        tail = newNode;
        size++;
    }
```

```java
    /*
     * RemoveL (Remove Last):
     * Deletes the last node of the list.
     *
     * Core idea:
     * - In a singly linked list, you must traverse to the node
     *   just before the tail.
     *
     * Steps:
     * 1. Handle empty and single-node cases.
     * 2. Traverse until temp.next == tail.
     * 3. Set temp as new tail.
     *
     * Invariant:
     * - tail.next must always be null.
     */
    public void RemoveL() {
        if (size == 0) {
            System.out.println("List is Empty");
            return;
        }

        if (size == 1) {
            head = tail = null;
            size--;
            return;
        }

        Node temp = head;

        while (temp.next != tail) {
            temp = temp.next;
        }

        temp.next = null;
        tail = temp;
        size--;
    }

    /*
     * PrintL:
     * Traverses the list from head to null.
     *
     * Core idea:
     * - Linked lists must be accessed sequentially.
     * - Stop traversal when node becomes null.
     */
    public void PrintL() {
        Node temp = head;
        while (temp != null) {
            System.out.println(temp.data + " ");
            temp = temp.next;
        }
    }
```

```java
    /*
     * Length:
     * Returns the number of nodes in the list.
     *
     * Core idea:
     * - Maintain size during insertions and deletions
     *   to avoid O(n) traversal every time.
     */
    public int Length() {
        return size;
    }

    public static void main(String[] args) {
        PracticeLinkedList list = new PracticeLinkedList();

        for (int i = 1; i <= 5; i++) {
            list.AddF(i);
        }

        list.PrintL();
        System.out.println("Size of LinkedList:- " + list.Length());

        list.AddAt(2, 100);

        System.out.println("---------------");
        list.PrintL();

        list.RemoveAt(2);
        System.out.println("------------------");
        list.PrintL();
    }
}
```

🧠 How to use these comments (important)

When revising:

- Read **only the comment**
- Close the code
- Try to **re-write the method from memory**

If you can do that, you truly understand linked lists.

If you want next, I can:

- Add **time & space complexity comments**
- Add **pointer diagrams as comments**
- Write the same level of detail for `Reverse`, `Middle`, `NthFromEnd`

Just tell me 👍